

IPL DATA ANALYSIS

Data Loading and Cleaning

```
In [92]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [ ]: ipl_df = pd.read_csv(r'E:\DS_Materials\Datasets\IPL_Dataset.csv')
ipl_df.head()
```

```
In [ ]: ipl_df.columns = ipl_df.columns.str.replace(' ', '_').str.strip().str.title()
ipl_df.head()
```

1. What is the maximum number of matches played by an individual player in a season. Print the player name along with the number of matched played.

```
In [ ]: max_matches_df = ipl_df['Matches'].max()
players_max_matches_df = ipl_df[ipl_df['Matches'] == max_matches_df]['Player'].tolist()

print(f"\nMaximum number of matches played by a player in a season: {max_matches_df}")
print(f"\nPlayers who played {max_matches_df} matches: {'', '.join(players_max_matches_df)}")
```

2. Top 2 players with maximum average who have scored atleast 2 half centuries.

```
In [ ]: top_avg_players_df = ipl_df[ipl_df['50'] >= 2].nlargest(2, 'Avg')[['Player', 'Avg']]
print(f"\nTop 2 players with maximum average who scored at least 2 half centuries:\n", top_avg_players_df)
```

3. Create 2 new columns based on player name. First column will have first name and second column will have last name. Eg: for the player Shikhar Dhawan, Shikhar will be the first name and Dhawan will be the last name.

```
In [ ]: ipl_df['Player'] = ipl_df['Player'].astype(str)
ipl_df[['First_Name', 'Last_Name']] = ipl_df['Player'].str.split(' ', n=1, expand=True)
ipl_df.head()
```

4. Create a new column (Cleaned_Highest_score) based on Highest score variable. Remove the Asterik(*) mark and convert the data type into INT.

```
In [ ]: ipl_df['Cleaned_Highest_Score'] = ipl_df['Highest_Score'].str.replace('*', '', regex=False).astype(int)
ipl_df.head()
```

5. Print the total number of centuries scored in the entire season.

```
In [ ]: total_centuries_df = ipl_df['100'].sum()
print(f"\nTotal number of centuries scored in the entire season: {total_centuries_df}")
```

6. Print all the player names whose strike rate is less than the average strike rate of all players in entire season. Print the player name, his strike rate and average strike rate.

```
In [ ]: average_strike_rate_df = round(ipl_df['Strike_Rate'].mean(), 2)

below_strike_players_df = ipl_df[ipl_df['Strike_Rate'] < average_strike_rate_df][['Player', 'Strike_Rate']]
below_strike_players_df['Average_Strike_Rate'] = average_strike_rate_df
below_strike_players_df = below_strike_players_df.sort_values(by='Strike_Rate', ascending=True)

print(f"\nPlayers with Strike Rate Below Average ({average_strike_rate_df}):\n", below_strike_players_df)
```

7. Please check the correlation between the features and create a heat map.

```
In [ ]: plt.figure(figsize=(12, 10))
numeric_df = ipl_df.select_dtypes(include=[np.number])
correlation_matrix = numeric_df.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Feature Correlation Heatmap')
plt.tight_layout()
plt.show()
plt.savefig('feature_correlation_heatmap.png')
plt.close()

print("\nCorrelation heatmap saved as 'feature_correlation_heatmap.png'")
```

8. Check the list of players who has an average greater than 50 as well strike rate above 120. Print player name, average and strike rate.

```
In [ ]: high_perf_df = ipl_df[(ipl_df['Avg'] > 50) & (ipl_df['Strike_Rate'] > 120)][['Player', 'Avg', 'Strike_Rate']]
high_perf_df = high_perf_df.sort_values(by='Avg', ascending=True)
print(f"\nPlayers with Avg > 50 & Strike Rate > 120:\n", high_perf_df)
```

9. Please check the list of players who has an average greater than 40 and balls faced above 100. Print player name, average and balls faced.

```
In [ ]: consistent_players_df = ipl_df[(ipl_df['Avg'] > 40) & (ipl_df['Balls_Faced'] > 100)][['Player', 'Avg', 'Balls_Faced']]
consistent_players_df = consistent_players_df.sort_values(by='Avg', ascending=True)
print(f"\nPlayers with Avg > 40 & Balls Faced > 100:\n", consistent_players_df)
```

10. Players who scored atleast one century in this season. Create visualization.

```
In [ ]: century_scorers_df = ipl_df[ipl_df['100'] >= 1].sort_values(by='100', ascending=False)
plt.figure(figsize=(10, 6))
sns.barplot(x='Player', y='100', data=century_scorers_df, hue='Player', palette='viridis', legend=False)
plt.title('Players who scored at least one century')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
plt.savefig('century_scorers.png')
plt.close()

print("\nPlayers who scored at least one century saved as 'century_scorers.png'")
```

11. Players who scored atleast 4 half centuries in this season.

```
In [ ]: half_century_players_df = ipl_df[ipl_df['50'] >= 4].sort_values(by='50', ascending=False)
print(f"\nPlayers with at Least 4 Half Centuries:\n", half_century_players_df[['Player', '50']])
```

12. Check the list of players who hit more than 45 boundaries and more than 10 sixes in this season.

```
In [ ]: boundary_hitters_df = ipl_df[(ipl_df['4S'] > 45) & (ipl_df['6S'] > 10)]
print(f"\nPlayers with More than 45 Boundaries & More than 10 Sixes:\n", boundary_hitters_df[['Player', '4S', '6S']])
```

13. Plot a histogram of number of matches played in a season by players.

```
In [ ]: plt.figure(figsize=(8, 5))
sns.histplot(ipl_df['Matches'], bins=10, kde=True)
plt.title("Histogram of Matches Played by Players")
plt.xlabel('Number of Matches')
plt.ylabel('Count of Players')
plt.tight_layout()
plt.show()
plt.savefig('matches_histogram.png')
plt.close()
```

14. Plot the histogram of balls faced by players.

```
In [ ]: plt.figure(figsize=(8, 5))
sns.histplot(ipl_df['Balls_Faced'], bins=10, kde=True, color='crimson')
plt.title("Histogram of Balls Faced by Players")
plt.xlabel('Number of Balls Faced')
plt.ylabel('Count of Players')
plt.tight_layout()
plt.show()
plt.savefig('balls_faced_histogram.png')
plt.close()
```

15. Top 10 players with most runs in a season.

```
In [ ]: top_run_scorers_df = ipl_df.nlargest(10, 'Runs')[['Player', 'Runs']]
print(f"\nTop 10 Players with Most Runs:\n", top_run_scorers_df)
```

16. Print the players who played the match but didn't get the batting.

```
In [ ]: non_batting_players_df = ipl_df[(ipl_df['Inns'] == 0) | (ipl_df['Balls_Faced'] == 0)][['Player', 'Matches']]
print(f"\nPlayers Who Played but didn't Bat:\n", non_batting_players_df)
```

17. Create a new column to show the percentage of total runs scored in 4s and 6s. Then print the top 5 players with maximum percentage.

```
In [ ]: ipl_df['Runs_in_4s'] = ipl_df['4S'] * 4
ipl_df['Runs_in_6s'] = ipl_df['6S'] * 6
ipl_df['Boundary_Runs'] = ipl_df['Runs_in_4s'] + ipl_df['Runs_in_6s']
ipl_df['Boundary_Percentage'] = (ipl_df['Boundary_Runs'] / ipl_df['Runs'] * 100).round(2)

ipl_df['Boundary_Percentage'] = ipl_df['Boundary_Percentage'].fillna(0)

top_boundary_percentage_df = ipl_df.sort_values(by='Boundary_Percentage', ascending=False).head(5)
print(f"\nTop 5 players with maximum percentage of runs in boundaries:")
print(top_boundary_percentage_df[['Player', 'Runs', 'Boundary_Runs', 'Boundary_Percentage']])
```

18. Print the players with top 5 Not Out percentages (Not Out percentage can be calculated as number of Not Outs divided by Innings).

```
In [ ]: ipl_df['Not_Out_Percentage'] = (ipl_df['Not_Out'] / ipl_df['Inns'] * 100).round(2)
ipl_df['Not_Out_Percentage'] = ipl_df['Not_Out_Percentage'].fillna(0)

top_not_out_percentage_df = ipl_df.sort_values(by='Not_Out_Percentage', ascending=False).head(5)
print(f"\nTop 5 players with highest Not Out percentages:")
print(top_not_out_percentage_df[['Player', 'Inns', 'Not_Out', 'Not_Out_Percentage']])
```

19. Create visualization of top 10 players with highest number of sixes.

```
In [ ]: top_six_hitters_df = ipl_df.sort_values(by='6S', ascending=False).head(10)
plt.figure(figsize=(12, 6))
sns.barplot(x='Player', y='6S', data=top_six_hitters_df, hue='Player', palette='plasma', legend=False)
plt.title('Top 10 Players with Highest Number of Sixes')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
plt.savefig('top_six_hitters.png')
plt.close()
```

20. Scatter plot of runs scored by a player v/s balls faced in a season. Then find the relationship between these 2 variables.

```
In [ ]: plt.figure(figsize=(10, 6))
sns.scatterplot(x='Balls_Faced', y='Runs', data=ipl_df, alpha=0.7)
plt.title('Scatter Plot of Runs vs Balls Faced')
plt.xlabel('Balls Faced')
plt.ylabel('Runs')

sns.regplot(x='Balls_Faced', y='Runs', data=ipl_df, scatter=False, color='red')

correlation = ipl_df['Runs'].corr(ipl_df['Balls_Faced'])
plt.annotate(f'Correlation: {correlation:.2f}',
            xy=(0.05, 0.95), xycoords='axes fraction',
            fontsize=12, bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="black"))

plt.tight_layout()
plt.show()
plt.savefig('runs_vs_balls_faced.png')
plt.close()
```

```
print(f"\nScatter plot of runs vs balls faced saved as 'runs_vs_balls_faced.png'")
print(f"Correlation between runs and balls faced: {correlation:.2f}")
```